

Reflection: Playwright Test Script Generator Agent

Project: Agentic Playwright Test Script Generator **Target:** Swag Labs (SauceDemo)

1. Where the agent genuinely added value

The main value of the agent was that it treated test creation as an iterative engineering task rather than a one-shot code-generation problem. It first explored the live Swag Labs application, used the observed page structure to plan the scenario, generated a Page Object Model (POM)-based Playwright test, and then attempted to execute the generated test. This creates a feedback loop between the application and the generated automation rather than relying only on assumptions made from a user story.

For example, a plain code-generation approach could produce a test using guessed selectors such as button text or CSS attributes without checking whether those selectors actually exist on the target application. The agent's exploration step provides runtime evidence that can be used to choose or validate locators before the test is executed.

The POM generation also added value. Instead of producing one large flat script, the agent organized the flow into reusable page objects for login, inventory, cart, and checkout. This makes the generated automation easier to maintain and provides a structure that can be reused when additional user stories are supplied.

2. Where a manual or plain approach would have been sufficient

For the four known Swag Labs scenarios, a manual implementation would have been faster and more deterministic. The application is simple, stable, and well suited to conventional Playwright automation. Once the required locators and workflow are known, a human can write the five tests directly and achieve passing results with little iteration.

A plain prompt-to-code model would also have been sufficient for generating an initial version of the scripts because the scenarios are clearly specified and the target application's structure is familiar. Therefore, the agent is not valuable merely because it can write TypeScript; the additional value comes from its ability to inspect, execute, interpret failures, and revise the generated automation.

3. Evidence from the implementation

Activity	Observed result	Value
Live exploration	Agent explored https://www.saucedemo.com/	Reduced reliance on guessed UI structure
Scenario planning	Mapped the supplied story to the checkout scenario	Converted natural-language intent into a concrete test flow
Code generation	Generated a POM-based Playwright test	Improved reuse and maintainability
Execution	Static Playwright suite: 5/5 tests passed	Validated the generated automation against the real application
Failure handling	Agent detected a Windows process-spawn failure ("spawn ENOENT")	Highlighted the importance of execution feedback; runner requirements

4. What I learned

The biggest lesson is that agentic automation is most useful when the environment is uncertain or when generated code must be validated against a live system. The execution failure also highlighted an important distinction: an agent can have an agentic architecture on paper, but the value is only real when the execution-and-repair loop is robust enough to respond to actual failures.

In this project, the successful Playwright tests demonstrate that the underlying automation is correct. The `spawn EINVAL` issue exposed a Windows-specific weakness in the agent runner rather than a test defect. Fixing that runner is therefore part of making the agent genuinely reliable.

5. Overall reflection

For a small, stable application like Swag Labs, the agent does not replace a skilled automation engineer. Manual scripting is faster for a handful of fixed scenarios. The agent becomes more valuable when the input changes frequently, when locators are unknown, when the target page must be inspected dynamically, or when generated tests need to be executed and repaired automatically.

Conclusion: The strongest contribution of the agent is not code generation itself. It is the closed-loop workflow: **explore** → **plan** → **generate** → **execute** → **inspect failure** → **repair** → **retry**. That loop is what distinguishes the solution from a conventional prompt-to-code generator.